

Evaluating Foundation Models Trained with Earth Observation

Data

Lillianne Thomas

Wei Ji Leong

Daniel Wiesmann

Soumya Ranjan

Evaluating Foundation Models Trained with Earth Observation Data

Topics Covered

- What are Earth Observation Foundation Models?
- Hands-on with Prithvi and Clay
- Flood detection case study: 2022 Pakistan floods

Learning Objectives

By the end of this chapter, you will be able to:

1. Explain what foundation models are and how they work
2. Use pre-trained EOFMs (Prithvi, Clay) to generate embeddings
3. Analyze embeddings using PCA and t-SNE
4. Train a classifier on top of foundation model embeddings
5. Compare different foundation models for the same task

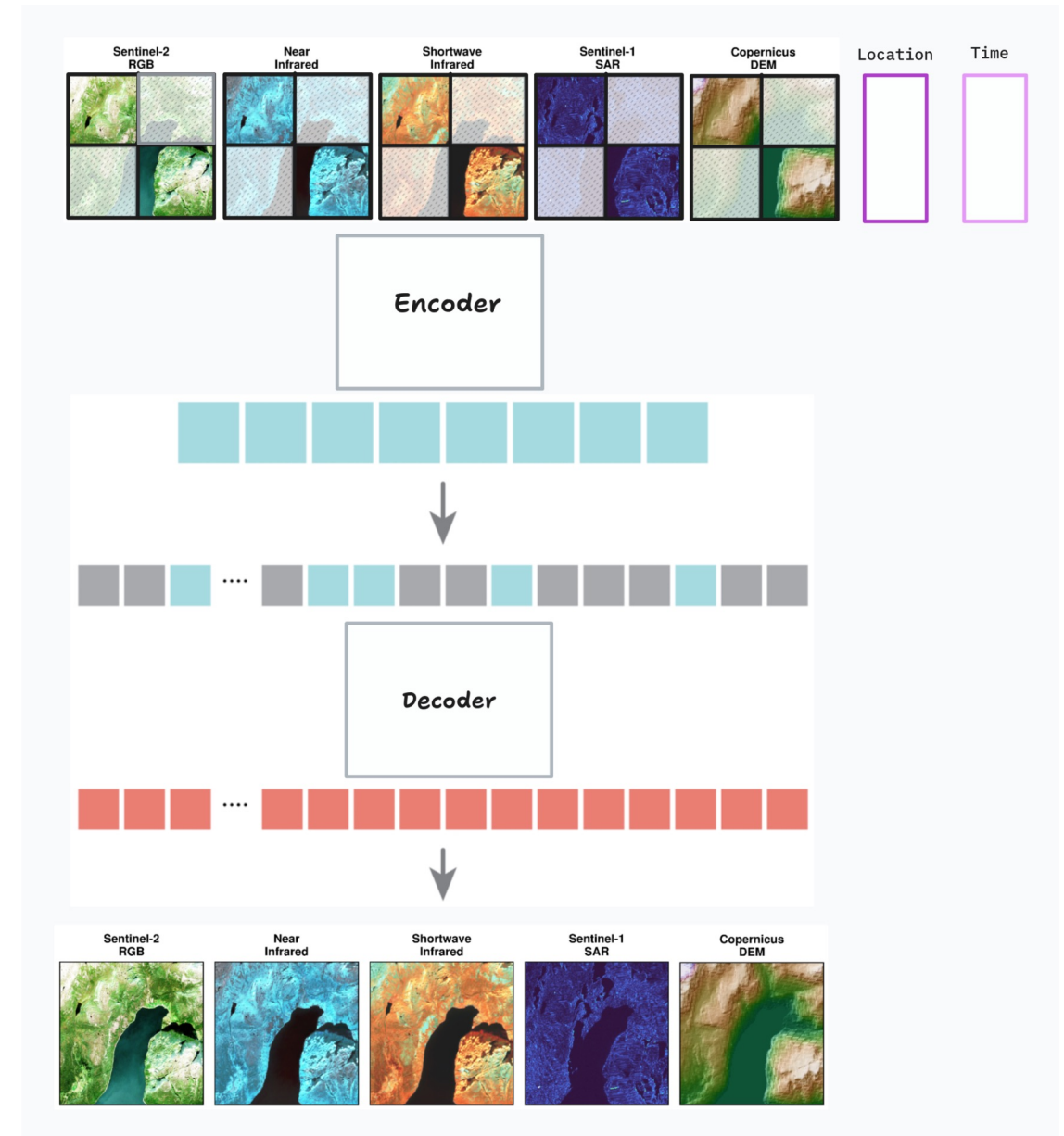
What is a Foundation Model?

Definition: Large-scale ML models pre-trained on vast amounts of remote sensing data to learn general-purpose representations

Key Characteristics:

- Pre-trained on diverse satellite imagery (optical, radar) or location attributes (elevation)
- Learn without human-annotated labels (self-supervised)
- Can be adapted to many downstream tasks
- Reduce time and cost of developing specialized applications

Examples: Prithvi, Clay, DOFA, Presto

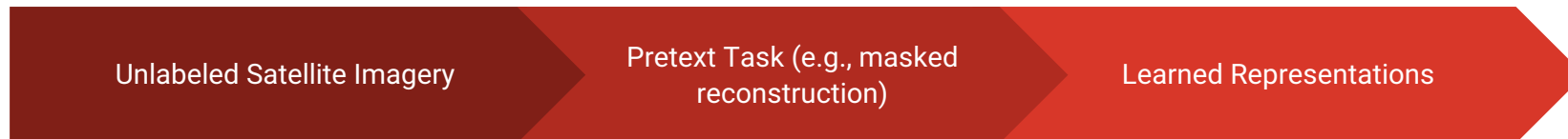


How Foundation Models Learn

Common Approaches:

- Masked Reconstruction — Hide parts of image, predict missing content
- Contrastive Learning — Distinguish similar vs. dissimilar pairs
- Generative Methods — Learn to synthesize input distributions

Key Insight: No labeled data required for pre-training!



How Foundation Models

Learn

How it works:

1. Divide image into fixed-size patches (e.g., 16×16 pixels)
2. Treat patches as a sequence of tokens
3. Apply self-attention to model relationships
4. Output: embeddings that capture representative features

Note: Vision Transformers (ViTs) are the backbone, not the foundation model itself



Why Foundation Models Matter

Benefit	Description
Reduced Training Costs	Fine-tune instead of training from scratch
Faster Development	Ready-to-use features and embeddings
Better with Limited Data	Robust even with small labeled datasets
Scalability	One model → many applications
Democratizing Access	Open-source, no expensive infrastructure
Better Backbones	Trained on EO data, not just RGB
90% with 10% Effort	Competitive results, fraction of the work

Limitations & Challenges

Current Limitations

Validation Gaps:

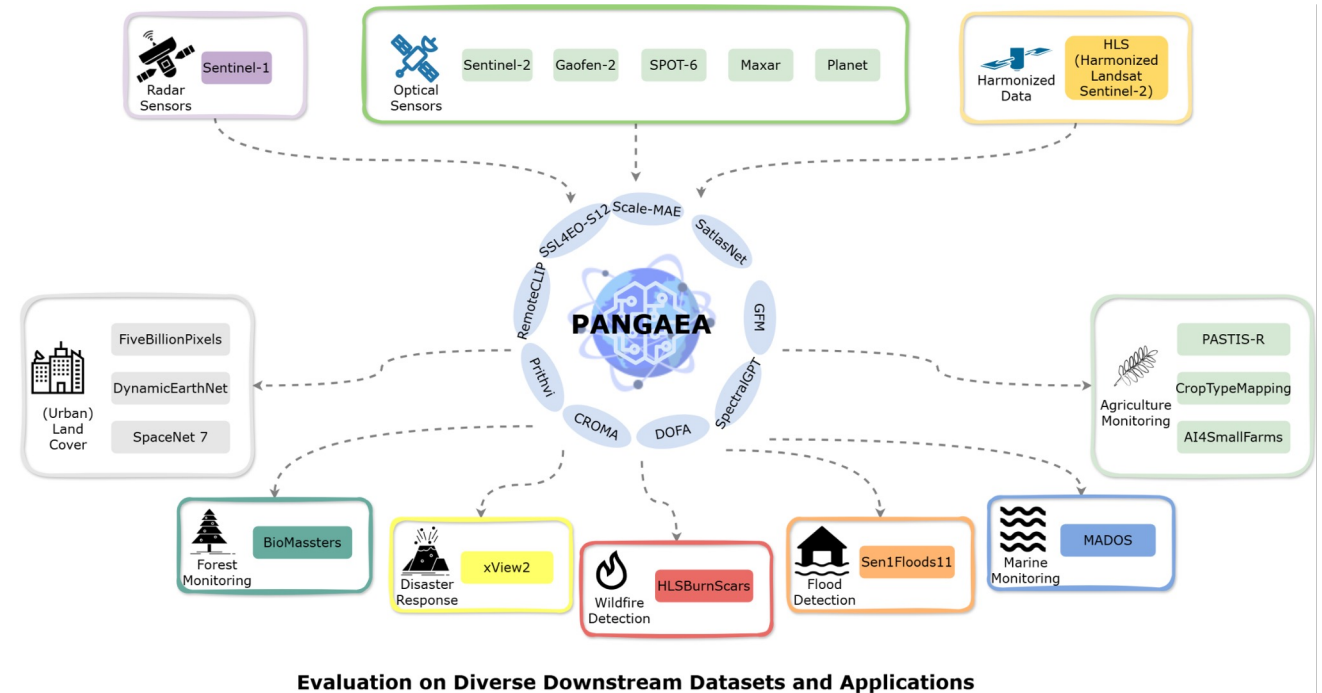
- Still relatively new in EO domain
- Need more validation across diverse applications

Domain-Specific Challenges:

- May struggle with under-represented conditions
- Rapidly changing environments (urban areas)
- Unique spectral signatures (polar regions)

EO-Specific Complexity:

- Diverse sensors, resolutions, wavelengths
- Temporal dynamics
- Metadata variations (CRS, view angles, providers)



Marsocci, V., Jia, Y., Bellier, G. L., Kerekes, D., Zeng, L., Hafner, S., Gerard, S., Brune, E., Yadav, R., Shibli, A., Fang, H., Ban, Y., Vergauwen, M., Audebert, N., & Nascetti, A. (2024). *PANGAEA: A Global and Inclusive Benchmark for Geospatial Foundation Models* (Version 1). arXiv. <https://doi.org/10.48550/ARXIV.2412.04204>

Case Study Introduction

2022 Pakistan Floods

The Disaster:

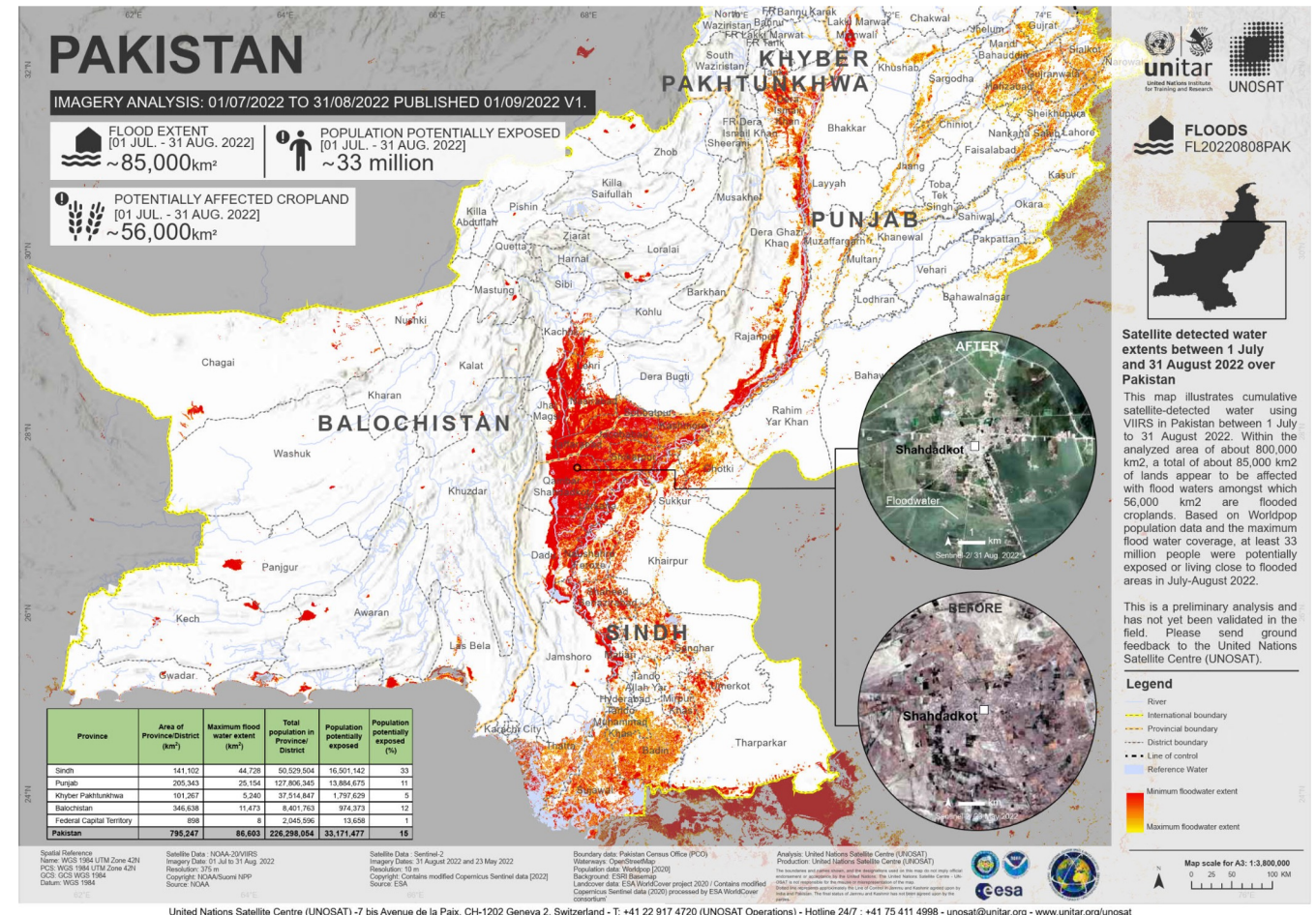
- Unprecedented monsoon rainfall from July-August 2022
- ~33 million people affected
- Vast agricultural lands submerged
- \$30+ billion in damages

Our Study Area:

- Padidan, Sindh Province, Pakistan
- One of the most severely impacted regions
- Indus River overflow

Our Goal:

- Use foundation models to detect and analyze flood conditions from satellite imagery



Satellite-detected water extent from UNOSAT (<https://unosat.org/products/3346>)

The Two Models

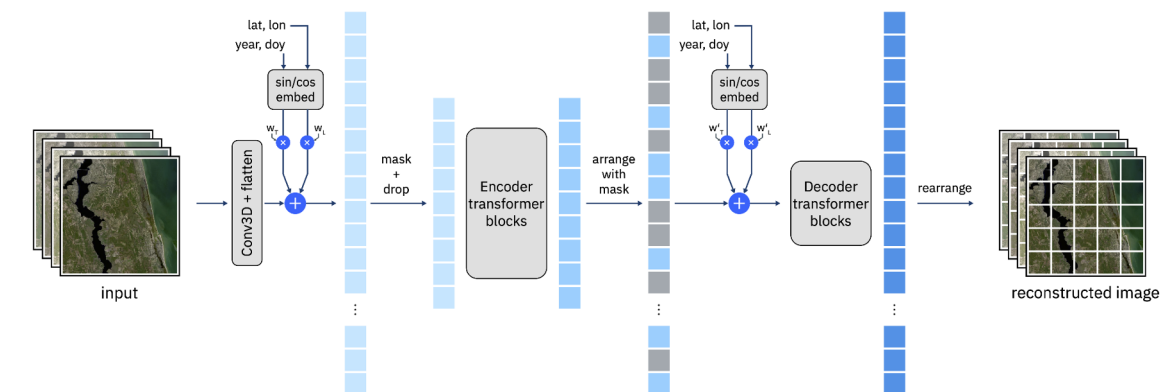
Feature	Prithvi-EO-2.0	Clay
Developers	NASA + IBM	Clay Foundation
Architecture	ViT + MAE	ViT-based
Training Data	HLS (Harmonised Landsat-Sentinel-2)	Multi-sensor
Input Format	Quadruplets (4 timesteps)	Single Images
Temporal	Built-in temporal modeling	Per-image
Flexibility	HLS-optimized	Variable resolution/sensors

Prithvi Architecture

Input: B, C, T, H, W (Bands, Channels, Time, Height, Width)

Key Features:

- Masked Autoencoder (MAE) self-supervised learning
- 3D patch embeddings for spatiotemporal features
- Temporal + spatial embeddings
- 6 spectral bands: Blue, Green, Red, NIR, SWIR1, SWIR2



Prithvi Architecture (<https://huggingface.co/ibm-nasa-geospatial/Prithvi-EO-2.0-300M>)

4 Images (T=4)

3D Patch Embedding

Transformer Encoder

CLS Token

Data Pipeline

1. Define Location & Dates → Padidan, Pakistan | June-October 2022
2. Query STAC Catalog → NASA CMR → HLS S30 Collection
3. Download Imagery → 22 images, 6 bands, 256×256 pixels
4. Format for Model → Create quadruplets (overlapping sequences of 4)
5. Generate Embeddings → 1024-dimensional vectors per quadruplet

Visualizing the Flood Progression

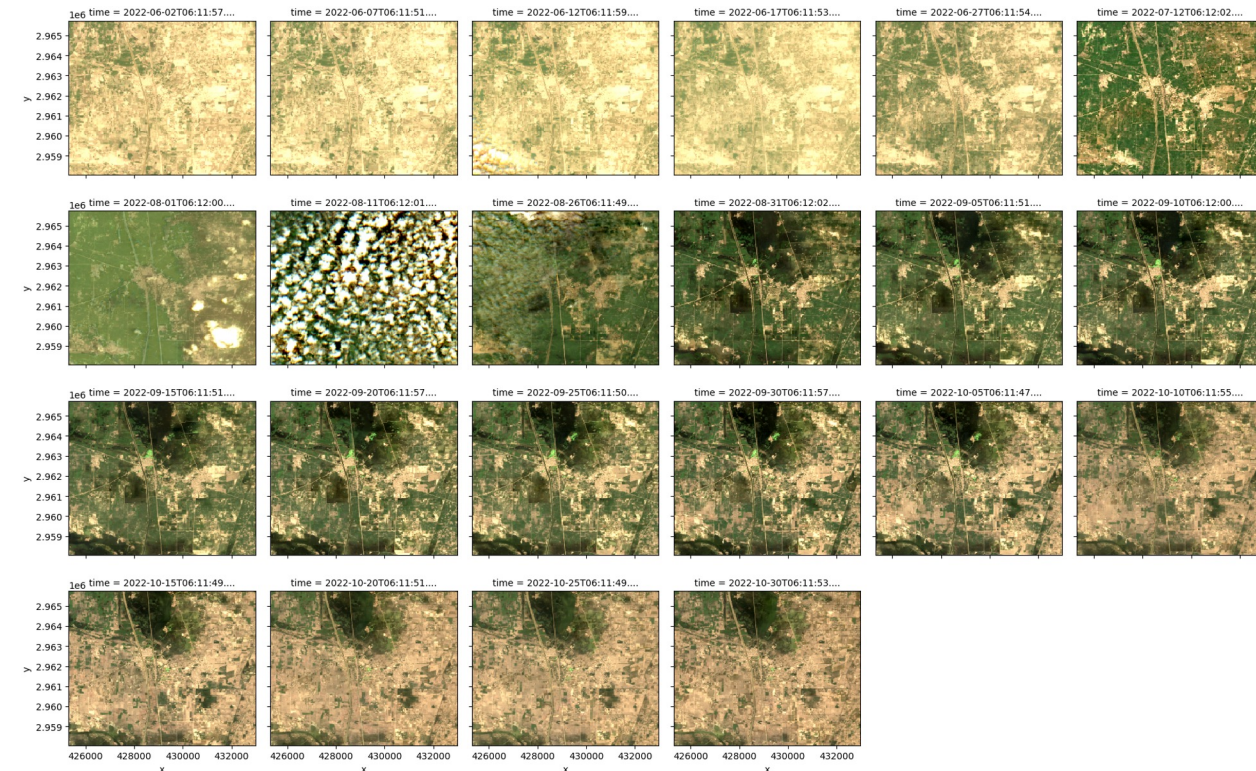
June 2022: Pre-Flood (Normal), 5 images

July-late August 2022: FLOOD, 4 images

Late August - October 2022: Post-Flood (Recovery), 13 images

What we see:

- Pre-flood: Agricultural fields, dry land
- During flood: Dark pixels (likely water inundation)
- Post-flood: Gradual recovery, some standing water



Dimensionality Reduction with PCA

Why PCA?

- Embeddings are 1024-dimensional
- PCA extracts directions of maximum variance
- Project to 1-2 dimensions for visualization

What we expect:

- Similar conditions → cluster together
- Different conditions → separate in PCA space

Quadruplet Embeddings

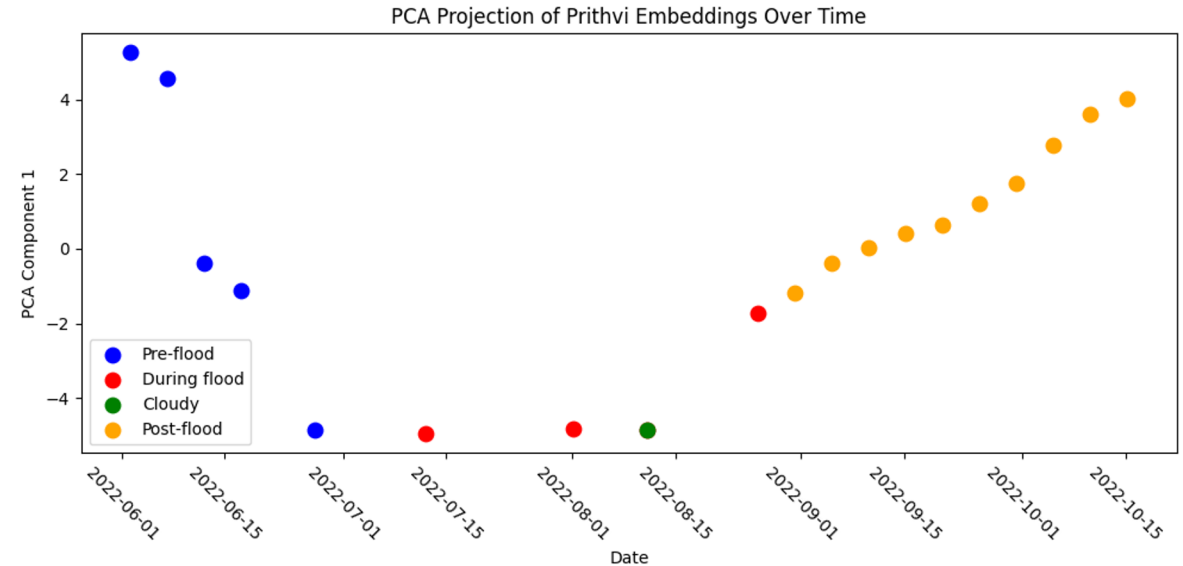
First Approach: CLS Token per Quadruplet

What we do:

- Pass each quadruplet through Prithvi
- Extract CLS token (summarizes all 4 timesteps)
- Get 19 embeddings (one per quadruplet)

Result:

- Some separation between flood phases
- But... there's an issue with how we are using these embeddings here...it is hard to cluster temporally when a given time step can exist in multiple quadruplets



Per-timestep Embeddings

Cleaner Analysis with Individual Timesteps

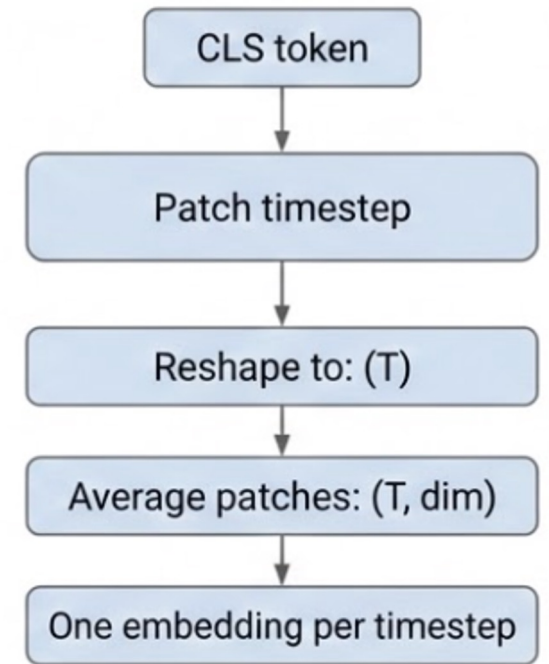
Prithvi outputs:

- CLS token (whole sequence summary)
- Patch tokens for each timestep ← We want to use these

Extraction:

- Embedding shape: $(1, \text{num_tokens}, \text{dim}) \rightarrow \text{Reshape to: } (T, \text{patches_per_timestep}, \text{dim}) \rightarrow \text{Average patches: } (T, \text{dim}) \rightarrow \text{One embedding per timestep}$

Result: 22 embeddings, one per image, no mixing!



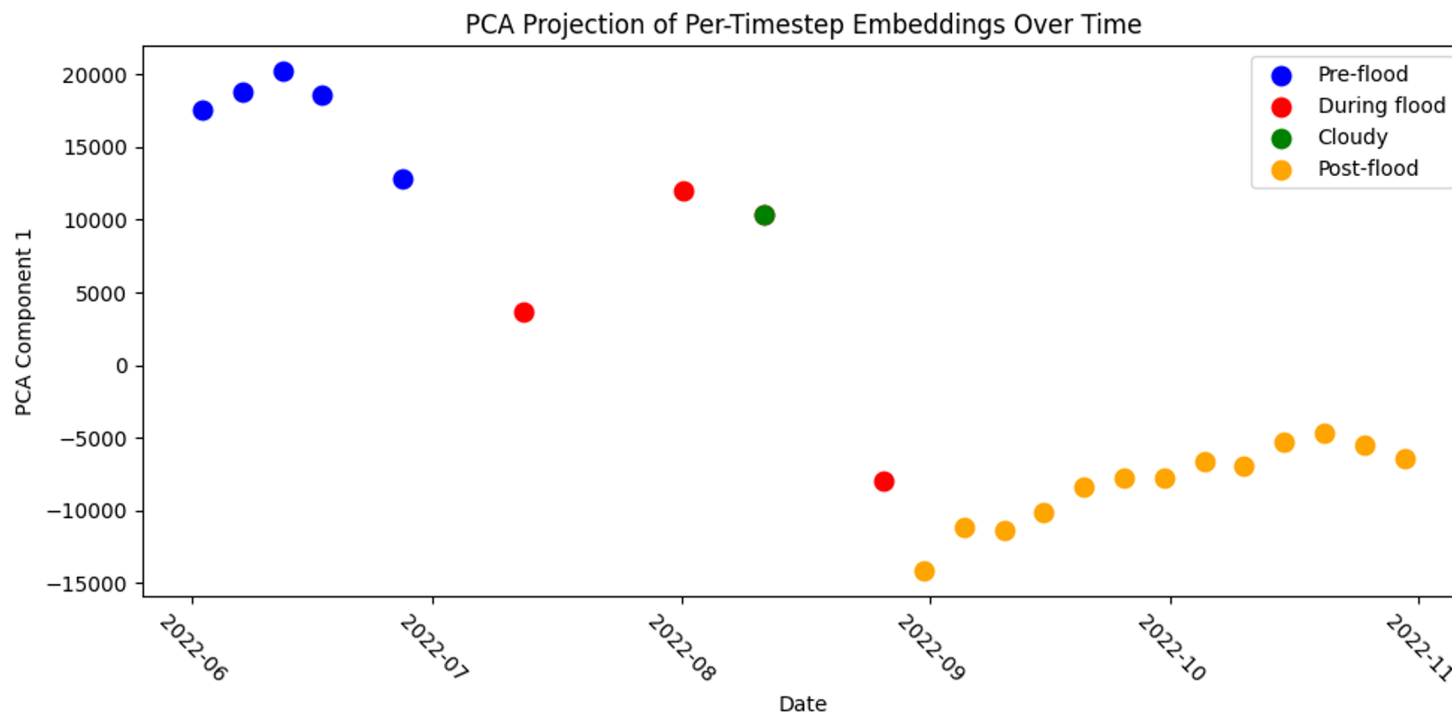
PCA Results (Per-Timestep)

Fairly Clear Separation of Flood Phases

PCA Component 1 vs Time

Key Insight:

- The model distinguishes flood phases without any labels!



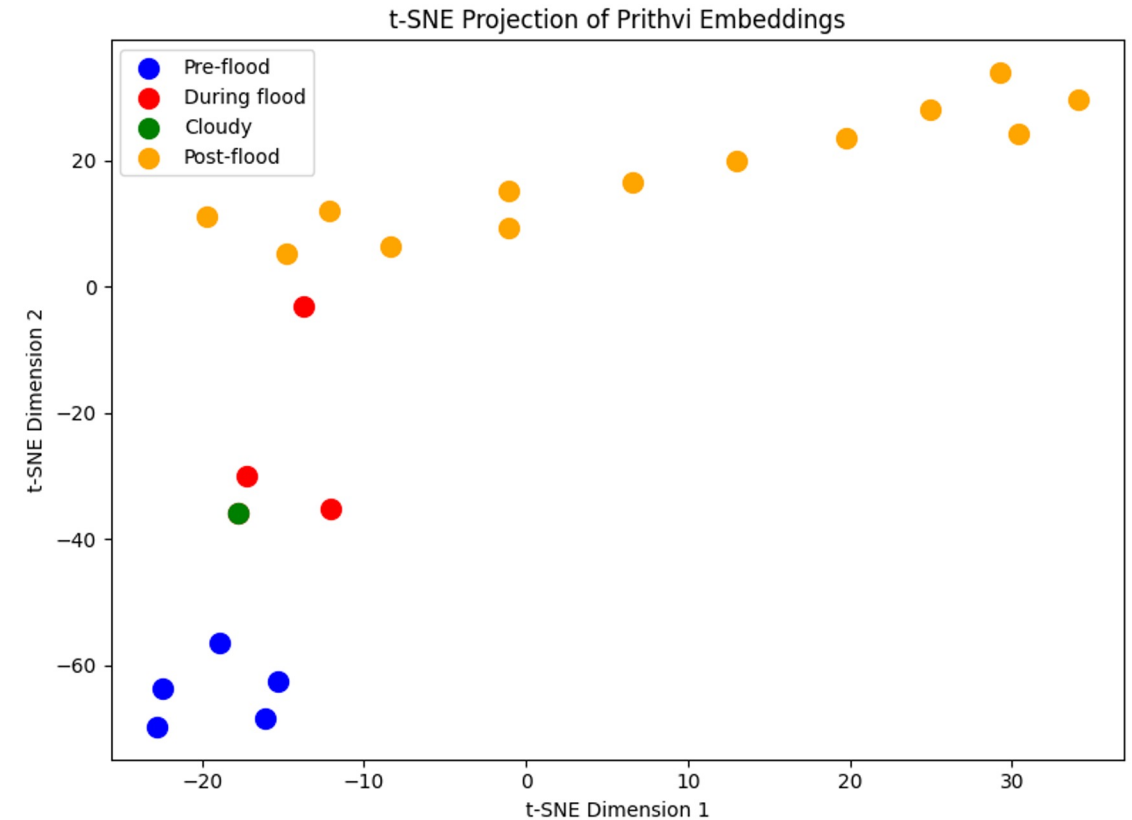
t-SNE Analysis

Preserving Local Structure

- t-SNE vs PCA:
 - PCA: Linear projection, preserves global variance
 - t-SNE: Non-linear, preserves local neighborhoods

Results:

- Confirms PCA findings
- Fairly distinct clusters for each flood phase



Training a Classifier

From Embeddings to Predictions

1. Use per-timestep embeddings as features
2. Assign labels: 0=cloudy, 1=pre-flood, 2=flood, 3=post-flood
3. Partition timesteps into training and testing sets
4. Train simple SVM classifier

Result: Strong classification performance with minimal data!

Clay Model

Key Difference:

- Clay processes individual images (not quadruplets)
- One image in → one embedding out
- No temporal mixing

For clay, embeddings are directly per-timestep

Prithvi

[img1, img2, img3, img4] → 1 embedding

Clay

[img1] → 1 embedding
[img2] → 1 embedding

Clay Results

Comparing the Models

PCA on Clay Embeddings:

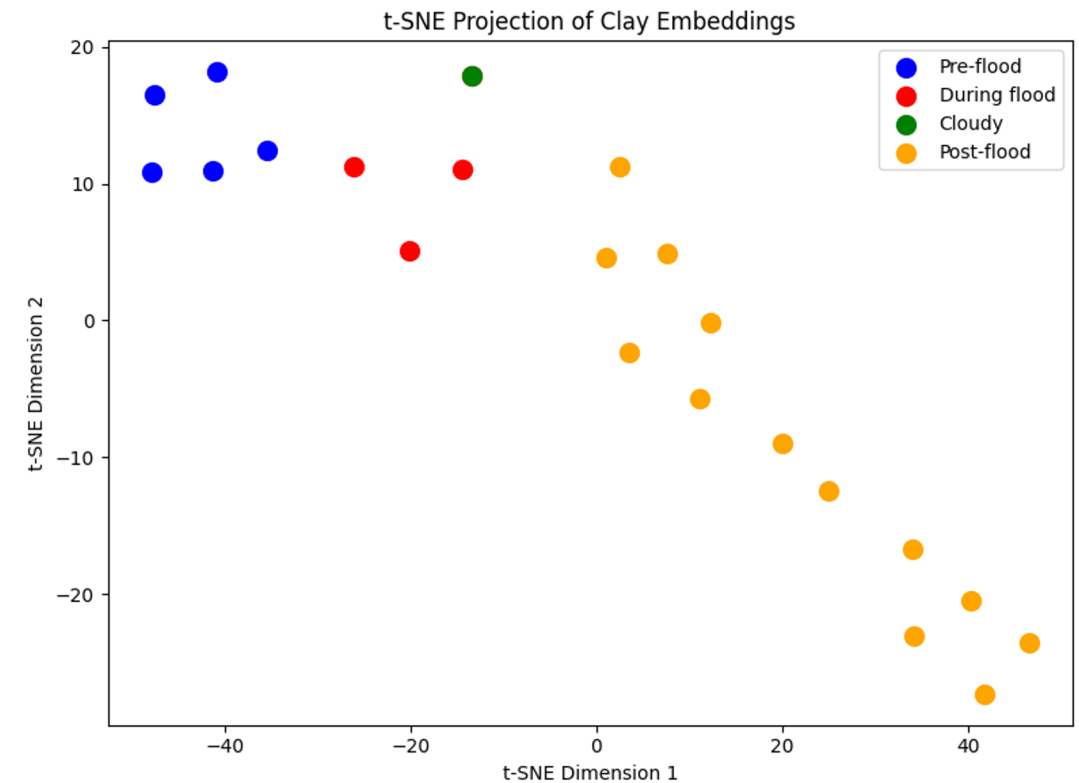
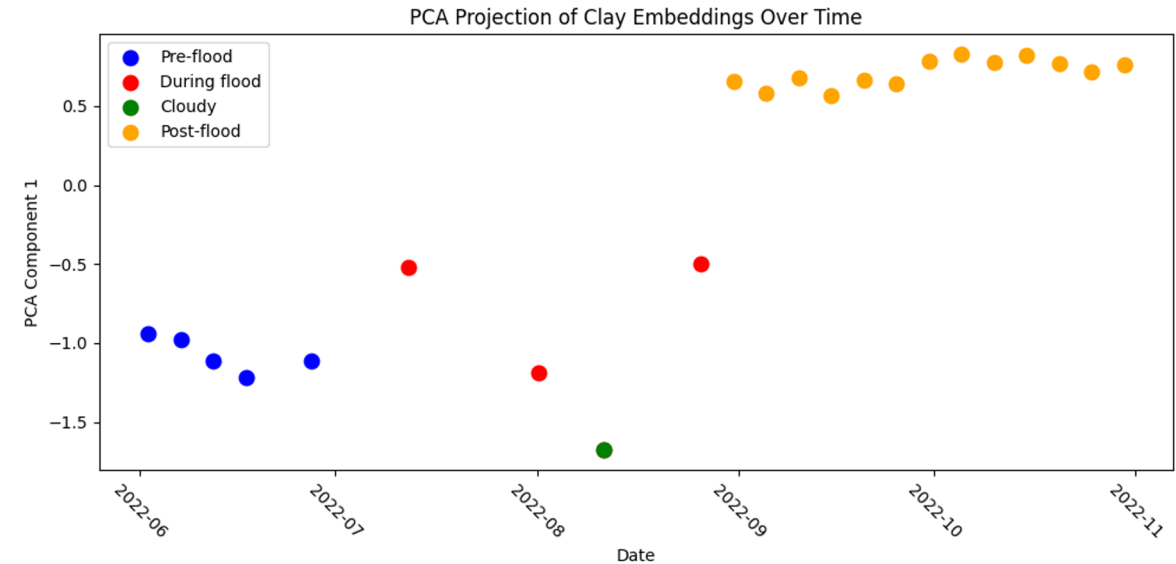
- Also shows clear flood phase separation
- Slightly different clustering pattern
- Emphasizes different aspects of the data

Key Insight:

Both models capture flood dynamics, but with different learned representations

Practical Implication:

Ensemble multiple models for more robust predictions



Model Comparison Summary

Aspect	Prithvi-EO-2.0	Clay
Input	Quadruplets	Single images
Temporal Context	4 timesteps	Per-image
Embedding Extraction	Need to extract per-timestep	Direct
Clustering Quality	Excellent	Excellent
Possibly best for	Temporal analysis	Flexible applications

Key Takeaways

What We Learned

1. Foundation models learn general representations without labels
2. Self-supervised learning enables training on vast unlabeled data
3. Embeddings capture meaningful patterns — flood phases cluster naturally
4. Different models, different strengths
5. Simple classifiers on top of embeddings achieve strong results
6. Practical applications are accessible with just a few lines of code

Limitations & Future Work

What to Keep in Mind

Current Limitations:

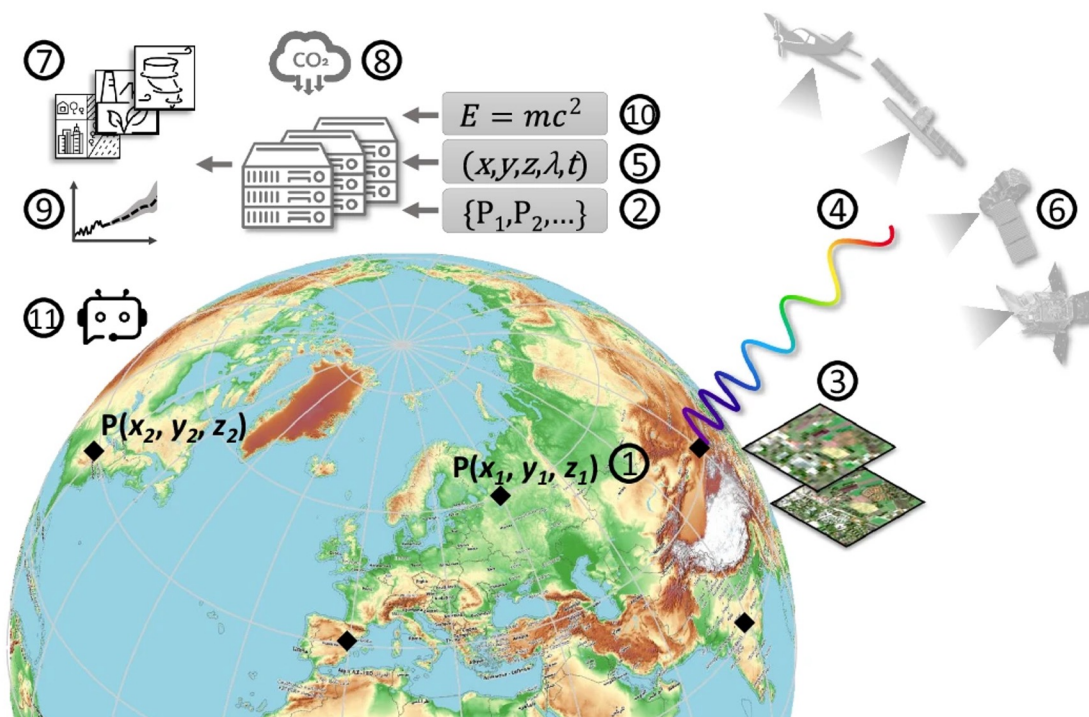
- Models may struggle with under-represented conditions
- Need validation across diverse geographies
- Computational requirements for inference

Future Directions:

- Multi-sensor fusion models
- Better temporal modeling
- Smaller, more efficient architectures
- Domain-specific fine-tuning

Essential Features

1. Geolocation embedding
2. Balanced geographical representations
3. Scale awareness
4. Wavelength embedding
5. Multisensory
6. The time variable
7. Task-agnostic
8. Carbon minimized
9. Uncertainty quantification
10. Physical consistency
11. Leveraging general-purpose VLMs/LLMs



Zhu, X. X., Xiong, Z., Wang, Y., Stewart, A. J., Heidler, K., Wang, Y., Yuan, Z., Dujardin, T., Xu, Q., & Shi, Y. (2026). On the foundations of Earth foundation models. *Communications Earth & Environment*, 7(1), 103.
<https://doi.org/10.1038/s43247-025-03127-x>

Resources

Learn More

Models:

- Prithvi: <https://github.com/NASA-IMPACT/Prithvi-EO-2.0>
- Clay: <https://clay-foundation.github.io/model>

Data:

- HLS: <https://hls.gsfc.nasa.gov/>
- Earth Data Login: <https://urs.earthdata.nasa.gov/>

Try It Yourself

Hands-On Exercise

The notebook includes:

- Complete code for both Prithvi and Clay
- Data download and preprocessing
- Embedding generation and visualization
- Classifier training

Experiment with:

- Different locations (your own AOI)
- Different time periods
- Different disaster types (fires, droughts)
- Other foundation models

Discussion

Key Questions to Consider:

- How would you adapt this workflow for a different disaster type?
- What are the trade-offs between Prithvi's temporal modeling and Clay's flexibility?
- How might you validate these results with ground truth data?
- What other downstream tasks could benefit from these embeddings?